
FONDAMENTI DI PROGRAMMAZIONE B

Tempo a disposizione: 2 ore

Nome Cognome Matricola

Esercizio 1 [C++] (15pt). Definire una classe templatica `Stack<T>` che realizza il tipo di dato astratto pila di elementi di tipo `T` con semantica LIFO (Last In, First Out). La classe deve definire:

- ▶ un costruttore senza argomenti che crea una pila vuota
- ▶ un metodo `void push(T elem)` che inserisce un elemento in cima alla pila
- ▶ un metodo `T pop()` che rimuove e restituisce l'elemento in cima; lancia un'eccezione se vuota
- ▶ un metodo `T top()` che restituisce l'elemento in cima senza rimuoverlo; lancia un'eccezione se vuota
- ▶ un metodo `bool isEmpty()` che dice se lo stack è vuoto o no

Sovraccaricare gli operatori `==` per confrontare se due pile sono uguali (stessi elementi nello stesso ordine) e `<<` per stampare la pila nel formato `[top -> e1, e2, ..., en]` (dove `e1` è l'elemento in cima).

Non è consentito utilizzare classi della STL. Se necessario, ridefinire gli opportuni metodi, costruttori e/o operatori. Specificare opportunamente eventuali metodi e parametri costanti. Massimizzare incapsulamento e information hiding.

Esercizio [Java] (15pt). Nel contesto della gestione di un laboratorio di robotica, si implementino le seguenti classi:

- ▶ Classe astratta `Robot` con identificativo (`String`) ed energia (`int`). La classe deve fornire costruttore, getter, equals (due robot sono uguali se hanno lo stesso identificativo) e dichiarare metodo astratto `int autonomia()` che rappresenta la capacità operativa del robot.
- ▶ Si implementino le classi `Drone` e `BraccioMeccanico` che estendono `Robot`. la classe `Drone` ha come attributo `quotaMassima`, un metodo `autonomia()` che ritorna l'autonomia calcolata come `energia + quotaMassima`. La classe `BraccioMeccanico` ha come attributo `pesoMaxSollevabile` e come il metodo `autonomia()` calcolata come `energia + peso`. entrambe devono fornire costruttore e ridefinire il metodo `equals`.
- ▶ Si implementi la classe `Laboratorio` caratterizzata da un nome e un insieme di robot. La classe deve fornire un costruttore senza parametri (laboratorio vuoto) e si deve implementare il metodo `aggiungiRobot(Robot r)` che aggiunge un robot. Se il robot è già presente deve lanciare un'eccezione controllata `RobotException` (da implementare). Implementare il metodo `int autonomiaTotale()` che ritorna la somma delle autonomie di tutti i robot. Implementare il metodo `Robot robotMigliore()` che restituisce il robot con autonomia massima, se il laboratorio è vuoto ritorna `Robot null`. Ridefinire il metodo `equals`: due laboratori sono uguali se hanno lo stesso nome e gli stessi robot (anche non nello stesso ordine).
- ▶ (+3pt) La classe `Robot` implementa `Comparable<Robot>` confrontando i robot in base alla loro autonomia.

N.B. Massimizzare incapsulamento e information hiding. Non è richiesta l'implementazione del metodo `hashCode` per le classi richieste. Per ciascuna classe, è possibile supporre di avere a disposizione una implementazione del metodo `hashCode` coerente col metodo `equals` che implementerete.